

AIRFLOW PIPELINES,
ENGINEERED

Airflow: DAG power, none of the daemon.

Why deploy a full service when you
only need a robust library?
Redefining Micro-Orchest...

+117k

downloads

**<50
MB**

RAM, total

SQLite

WAL state

THE CONTEXT

Where it hurts today

A metadata DB to babysit

DAGs drift from reality

CI for DAGs is painful

THE SHIFT

What you gain



Code-first

Pipelines are Python — versionable, testable.



Resume, don't restart

Every step checkpoints to the WAL.

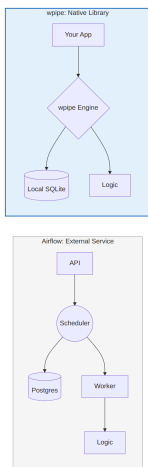


Featherweight

<50 MB. No broker, no daemon.

THE ARCHITECTURE

One pipeline, fully instrumented



State, retries and audit — inside a single run.

SIDE BY SIDE

wpipe vs. the status quo

Dimension	Apache Airflow	wpipe
Architecture	Client-Server (Centralized)	Native Library (Decentralized)
Footprint	GBs (Multiple containers)	MBs (Single process)
Persistence	External Postgres/Redis	SQLite WAL (Zero-Config)
Task Latency	Seconds (Scheduling overhead)	Milliseconds (Direct execution)

SEE IT IN ACTION

Six lines. That's it.

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3,
retry_delay=2)
def job(data):
    return {"ok":
run(data["job"])}

pipe =
Pipeline(pipeline_name="airflow_like")
```

CHECKPOINTS · RETRIES · TIMEOUT ·
PARALLEL · TRACKER

Try wpipe in your next pipeline ✓

Install the library, wrap your step
— checkpoints and docs appear
by themselves.

```
pip install wpipe
```

wpipe

Python-native orchestration